MINOR ASSIGNMENT

Course Name	Containerization for Developers
Course Code	CSE 4630

1. A WordPress container and a MariaDB container are running. WordPress uses the MariaDB container's IP address as WORDPRESS_DB_HOST. Both initially work correctly. The MariaDB container uses a named volume db-data. After MariaDB is removed and recreated using the same volume, WordPress can no longer connect to the database, although MariaDB is running and the data is intact.

a. Diagnose the problem and modify the deployment.

b. Ensure WordPress continues to connect even if MariaDB is recreated with a different IP address.

c. Preserve the existing database data.

d. Ensure MariaDB is accessible to WordPress but is not exposed unnecessarily to the host.

e. Give the Docker commands required to verify your solution.

Explain why your corrected configuration continues to work after the MariaDB container is replaced.

2. Create a Compose application containing an Alpine container that reads a password from a Docker secret.

Create the following file:

```
password.txt

MySecurePassword123
```

The container must:

- receive the secret named db_password
- read it from /run/secrets/db_password
- print the password
- terminate after printing it

3. Create two images demonstrating CMD and ENTRYPOINT.

```
# Image A
FROM ubuntu:20.04
CMD ["echo", "Hello from CMD"]

# Image B
FROM ubuntu:20.04
ENTRYPOINT ["echo"]
```

a. Build both images.

b. Run each without arguments.

c. Run each with an additional message.

d. Record the output.

e. Explain why the outputs differ.

f. State when CMD and ENTRYPOINT are appropriate.

4. Investigate the CPU architecture of the Docker environment and an image.

a. Determine Docker's architecture.

b. Determine architectures supported by Alpine.

c. Inspect a local image architecture.

d. Run Alpine.

e. Explicitly request linux/amd64.

f. Record and explain observations.

5. Create a simple custom Docker image and demonstrate the complete image lifecycle.

```
docker-command-assignment/
|
|-- Dockerfile
`-- index.html
```

Create the required files and demonstrate the relevant stages of the image/container lifecycle, including building the image, inspecting it, running a container, verifying the application, viewing the created resources, stopping/removing the container, and managing the image.

6. A developer was asked to create a lightweight Alpine-based Docker image for a Python application. The application requires only Python and curl at runtime. The existing image works, but it contains unnecessary packages and package-management data. Analyze the supplied Dockerfile, build it, identify unnecessary components, and redesign it to produce a more appropriate runtime image.

The developer's Dockerfile is:

```
FROM alpine
RUN apk update
RUN apk add python3
RUN apk add curl
RUN apk add bash
RUN apk add nano
RUN apk add build-base
WORKDIR /application
COPY app.py .
CMD ["python3", "app.py"]
```

- Build the original image as application-original:v1.
- Record its image size and examine its image history.
- Determine which installed packages are actually required at runtime.
- Redesign the Dockerfile so Python and curl remain available, Bash and Nano are removed, package indexes are not unnecessarily retained, and build-base is not retained in the runtime environment.
- Build the redesigned image as application-optimized:v1.
- Run both versions and verify that the application behaves identically.
- Compare image sizes and histories and explain why the optimized version is more appropriate.

Use the following app.py:

```
import platform
print("Application started successfully.")
print("Python version:", platform.python_version())
print("Container optimization test completed.")
```

7. A department is moving from one web container to several independent services. It has two identical web backends that should share traffic and a separate API service. The public system should have a single entry layer. The administrator also wants to understand how the architecture would evolve if services are frequently added or removed and HTTPS certificate management must later be automated. First implement a classroom load-balancing solution with HAProxy and two backend containers. Then explain how the same environment would conceptually change if Traefik were selected for dynamic routing and certificate automation. Your answer must discuss service-name discovery, published ports, Docker networks, static proxy configuration versus labels, Docker socket access, and TLS termination.

8. A college application requires a database that must preserve student records even if its database container is deleted and recreated. Deploy a MariaDB or PostgreSQL database using Docker, configure persistent storage, and create a database containing a students table with at least five records.

Then:

a. Verify the stored records using the database client.

b. Stop and remove the database container.

c. Recreate the container using the same persistent storage and demonstrate that the records are retained.

d. Create a logical backup of the database.

e. Explain the roles of the container, image, volume, environment variables, and backup file in your solution.

f. Record the commands/configuration used and the output before and after container recreation.

9. Develop a small application using Node.js, Python, Java, PHP, or Ruby that performs a meaningful task such as providing an HTTP service, processing a file, or retrieving and displaying structured information. Package the application as a Docker image.

Your solution must:

a. Select an appropriate official language runtime image.

b. Create a Dockerfile using suitable WORKDIR, COPY, RUN, USER, and CMD instructions wherever applicable.

c. Install at least one external dependency/package required by the application.

d. Build and execute the application entirely inside a container.

e. Create a second version using a smaller/slimmer base image where available and compare the resulting image sizes.

f. Run one version with explicit CPU and memory limits and observe its resource usage using docker stats.

g. Explain how Docker makes the application's runtime and dependencies reproducible across different computers.